

Índex

- 0. TEORIA
- 1. Skeletons
 - Per Vertex (Class A)
 - Per Fragment (Class B)
 - Per Geometry (Class C)
- 2. Receptes
 - Per Vertex
 - Per Fragment
 - Per Geometry
- 3. Exercises
 - Per Vertex
 - Per Fragment
 - Per Geometry



0. TEORIA

0.1 La Lògica Universal (Els 5 Passos)

Aquesta és la seqüència **CANÒNICA** per transformar qualsevol geometria (vèrtexs o objectes enters) respecte al seu propi centre o un punt arbitrari.

L'ordre matemàtic és SAGRAT:

1. **Centrar:** Portar el punt de referència al $(0, 0, 0)$.
2. **Esgalar:** Fer-ho gran o petit (sempre es fa des de l'origen).
3. **Rotar:** Girar (sempre es gira respecte a l'origen).
4. **Descentrar:** Tornar el punt de referència al seu lloc original.
5. **Moure (Translació Final):** Aplicar el moviment final (animació, explosió, etc.).

```
// Suposem que tens:  
// 'pos': la posició del vèrtex  
// 'C': el centre de l'objecte (BoundingBox Center o Baricentre)  
// 'scale': float (ex: 2.0 per doble, 0.5 per meitat)  
// 'rotMatrix': matriu de rotació  
// 'desplacament': vector de moviment final  
  
// 1. CENTRAR
```

```
pos = pos - C;

// 2. ESCALAR
pos = pos * scale;

// 3. ROTAR
pos = rotMatrix * pos; // En GLSL la matriu va a l'ESQUERRA

// 4. DESCENTRAR (Restaurar posició original relativa)
pos = pos + C;

// 5. MOURE (Translació final / Explosió)
pos = pos + desplaçament;
```

0.2 Matrius de Rotació

Copia aquestes funcions si necessites rotar manualment. Recorda que **angle** ha d'estar en **radians**.

Rotació Eix X

```
mat3 rotateX(float angle) {
    float c = cos(angle);
    float s = sin(angle);
    return mat3(
        1.0, 0.0, 0.0,
        0.0,  c, -s,
        0.0,  s,  c
    );
}
```

PROF

Rotació Eix Y

```
mat3 rotateY(float angle) {
    float c = cos(angle);
    float s = sin(angle);
    return mat3(
        c, 0.0, s,
        0.0, 1.0, 0.0,
        -s, 0.0, c
    );
}
```

```
mat3 rotateZ(float angle) {  
    float c = cos(angle);  
    float s = sin(angle);  
    return mat3(  
        c,  -s,  0.0,  
        s,   c,  0.0,  
        0.0, 0.0, 1.0  
    );  
}
```

0.3 Definició de Colors

Snippet ràpid per tenir la paleta de colors bàsica a mà.

```
const vec4 RED      = vec4(1.0, 0.0, 0.0, 1.0);  
const vec4 GREEN    = vec4(0.0, 1.0, 0.0, 1.0);  
const vec4 BLUE     = vec4(0.0, 0.0, 1.0, 1.0);  
const vec4 CYAN     = vec4(0.0, 1.0, 1.0, 1.0);  
const vec4 MAGENTA  = vec4(1.0, 0.0, 1.0, 1.0);  
const vec4 YELLOW   = vec4(1.0, 1.0, 0.0, 1.0);  
const vec4 WHITE    = vec4(1.0, 1.0, 1.0, 1.0);  
const vec4 BLACK    = vec4(0.0, 0.0, 0.0, 1.0);  
const vec4 GREY     = vec4(0.5, 0.5, 0.5, 1.0);  
const vec4 ORANGE   = vec4(1.0, 0.5, 0.0, 1.0);  
const vec4 PURPLE   = vec4(0.5, 0.0, 0.5, 1.0);  
const vec4 VIOLET   = vec4(0.58, 0.0, 0.82, 1.0);  
const vec4 PINK     = vec4(1.0, 0.41, 0.7, 1.0);
```

0.4 Càlcul de la Normal

1. Concepte Matemàtic: El Producte Vectorial

Per calcular una normal necessites **dos vectors** que estiguin sobre la superfície (i).

El producte vectorial (**Cross Product**) et dona un tercer vector perpendicular a tots dos.

La Regla d'Or: Per a qualsevol polígon pla (sigui un triangle, un quadrat o un pentàgon), **tota la superfície té la mateixa normal**.

- **Triangle:** Fas servir els seus 3 vèrtexs.
 - **Quadrat:** Aganfes 3 vèrtexs qualssevol (fan una cantonada) i calcules la normal com si fos un triangle. La normal resultant val per a tot el quadrat.
-

2. Opció A: Càlcul al Geometry Shader (Manual)

Aquesta és la manera clàssica i robusta. Calcles la normal abans d'emetre els vèrtexs i l'envies al FS.

Cas 1: La Base (Un Triangle)

Tens 3 punts: .

1. Calcula el vector del costat 1 ().
2. Calcula el vector del costat 2 ().
3. Fes el cross product.

```
vec3 V0 = gl_in[0].gl_Position.xyz;
vec3 V1 = gl_in[1].gl_Position.xyz;
vec3 V2 = gl_in[2].gl_Position.xyz;

// Vectors arestes
vec3 edge1 = V1 - V0;
vec3 edge2 = V2 - V0;

// Normal
vec3 N = normalize(cross(edge1, edge2));

// ATENCIÓ: Si el triangle es dibuixa en sentit anti-horari (CCW),
// la normal surt cap a fora.
```

Cas 2: Una Paret o Quadrat (4 Punts)

Tens 4 punts () formant un pla. No cal complicar-se.

Estratègia: Ignora el 4t punt. Fes servir i i aplica la mateixa fórmula del triangle.

Si estàs fent una **extrusió** (prisma), sovint és més fàcil pensar en vectors "físics":

- Vector **Horitzontal** (el terra).
- Vector **Vertical** (la paret).

```
// Suposem que estem fent una paret entre 'base' i 'top'
vec3 v_base_curr = ...;
vec3 v_base_next = ...;
vec3 v_top_curr = ...;

// 1. Vector Horitzontal (terra)
vec3 horitzontal = v_base_next - v_base_curr;

// 2. Vector Vertical (paret amunt)
vec3 vertical = v_top_curr - v_base_curr;
```

```
// 3. Normal (Terra x Paret = Enfora)
vec3 NormalParet = normalize(cross(horitzontal, vertical));

// Assignem aquesta normal a TOTS els 4 vèrtexs del quadrat
gNormal = NormalParet;
```

3. Opció B: Càlcul al Fragment Shader (Automàtic)

Aquesta opció és perfecta per a figures de cares planes (Low Poly, Prismes, Cubs) si no vols calcular res al GS.

Com funciona: La GPU mira la posició del píxel veí i dedueix la inclinació de la cara.

GS:

Només envia la posició (**gPos**). No cal calcular normals.

FS:

Fes servir les derivades **dFdx** i **dFdy**.

```
#version 330 core

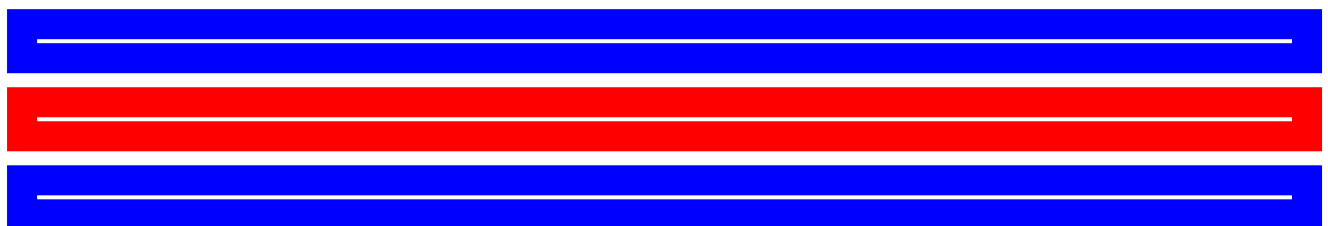
in vec3 gPos; // Posició en Eye Space (interpolada)
out vec4 fragColor;

void main() {
    // CALCULAR ELS VECTORS TANGENTS AUTOMÀTICAMENT
    vec3 dx = dFdx(gPos); // Com canvia la posició cap a la dreta
    vec3 dy = dFdy(gPos); // Com canvia la posició cap amunt

    // CALCULAR LA NORMAL
    // El producte vectorial de les derivades és la normal de la
    superfície plana
    vec3 N = normalize(cross(dx, dy));

    // ... Il·luminació normal ...
    fragColor = vec4(N.z, N.z, N.z, 1.0);
}
```

PROF



1. Skeletons

Class A-Esqueleto (PER-VERTEX) (Deformación)

A-Vertex Shader

```
#version 330 core

// --- INPUTS (from your 3D model) ---
layout (location = 0) in vec3 vertex;
layout (location = 1) in vec3 normal;
layout (location = 2) in vec3 color;
layout (location = 3) in vec2 texCoord;

// --- OUTPUTS (to the Fragment Shader) ---
out vec4 frontColor; // El color FINAL calculado por vértice
out vec2 vtexCoord; // La coordenada de textura

// --- UNIFORMS (from the viewer) ---
uniform mat4 modelViewProjectionMatrix;
uniform mat3 normalMatrix;

void main()
{
    vec4 vertex_objectspace = vec4(vertex, 1.0);
    vec3 normal_objectspace = normal;

    // Calcula la normal en Eye Space
    vec3 N = normalize(normalMatrix * normal_objectspace);

    // Calcula el color (iluminación simple por Z)
    frontColor = vec4(color, 1.0) * N.z;

    // Pasa la coordenada de textura
    vtexCoord = texCoord;

    // Calcula la posición final
    gl_Position = modelViewProjectionMatrix * vertex_objectspace;
}
```

PROF

NOTA Si te pide modificar la Projection y la Normal Space, se tiene que calcular por si solo:

```
#version 330 core

// --- INPUTS (from your 3D model) ---
layout (location = 0) in vec3 vertex;
layout (location = 1) in vec3 normal;
layout (location = 2) in vec3 color;
layout (location = 3) in vec2 texCoord;
```

```
// --- OUTPUTS (to the Fragment Shader) ---
out vec4 frontColor; // El color FINAL calculado por vértice
out vec2 vtexCoord; // La coordenada de textura

// --- UNIFORMS (from the viewer) ---
uniform mat4 modelViewProjectionMatrix;
uniform mat3 normalMatrix;

void main()
{
    //P_object = texture ...
    //P_eye_4 = modelViewMatrix * vec4(P_object, 1.0);

    // Calcula la posición final
    gl_Position = projectionMatrix * P_eye_4;
}
```

A-Fragment Shader

```
#version 330 core

// --- INPUT (from the Vertex Shader) ---
in vec4 frontColor; // Recibe el color interpolado

// --- OUTPUT ---
out vec4 fragColor;

void main()
{
    // Simplemente asigna el color calculado en el VS
    fragColor = frontColor;
}
```

PROF

Class B-Esqueleto (PER-FRAGMENT)

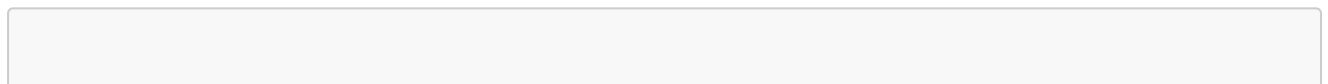
En esta categoría el trabajo duro se hace en el **Fragment Shader**. Hay dos variantes principales según lo que pida el examen: trabajar con imágenes (B1) o calcular física de la luz (B2).

B1: Texturas (Mapeado de Imágenes)

Úsalo cuando: El examen pida "Texturizar el objeto", "Invaders", "Kong", "Mostrar una imagen".

- **Clave:** Pasar `texCoord` al FS y usar `texture()`.

B1-Vertex Shader



```
#version 330 core

layout (location = 0) in vec3 vertex;
layout (location = 3) in vec2 texCoord; // Necesitamos esto sí o sí

out vec2 vtexCoord; // Lo pasamos al Fragment Shader

uniform mat4 modelViewProjectionMatrix;

void main()
{
    // Pasamos la coordenada tal cual
    vtexCoord = texCoord;

    // Posición estándar
    gl_Position = modelViewProjectionMatrix * vec4(vertex, 1.0);
}
```

B1-Fragment Shader

```
#version 330 core

in vec2 vtexCoord;
out vec4 fragColor;

uniform sampler2D colorMap; // La textura cargada

void main()
{
    // Muestreamos el color de la textura en la coordenada (s,t)
    vec4 texColor = texture(colorMap, vtexCoord);

    // OPCIONAL: Descartar píxeles transparentes (Alpha Testing)
    // if(texColor.a < 0.5) discard;

    fragColor = texColor;
}
```

PROF

B2: Iluminación (Modelo Phong / Alta Calidad)

Úsalo cuando: El examen pida "Phong Shading" (no Gouraud), "Especcularidad", "Brillos", o "Iluminación por fragmento".

- **Clave:** Pasar la **Posición (P)** y la **Normal (N)** interpoladas al FS. El color se calcula abajo, no arriba.

B2-Vertex Shader

```
#version 330 core

layout (location = 0) in vec3 vertex;
layout (location = 1) in vec3 normal;

out vec3 P; // Posición del vértice en View Space
out vec3 N; // Normal del vértice en View Space

uniform mat4 modelViewMatrix;
uniform mat3 normalMatrix;
uniform mat4 modelViewProjectionMatrix;

void main()
{
    // 1. Calculamos P en View Space (necesario para vectores de luz)
    P = (modelViewMatrix * vec4(vertex, 1.0)).xyz;

    // 2. Calculamos N en View Space (normalizada)
    N = normalize(normalMatrix * normal);

    // 3. Posición en pantalla (Clip Space)
    gl_Position = modelViewProjectionMatrix * vec4(vertex, 1.0);
}
```

B2-Fragment Shader

```
#version 330 core

in vec3 P; // Interpolado (Posición del píxel)
in vec3 N; // Interpolado (Normal del píxel)
out vec4 fragColor;

// Propiedades de la luz y el material
uniform vec4 lightPosition; // Suele venir en View Space
uniform vec4 matAmbient, matDiffuse, matSpecular;
uniform float matShininess;

void main()
{
    // 1. Renormalizamos N (la interpolación altera la longitud)
    vec3 N_norm = normalize(N);

    // 2. Calculamos vectores L (Luz), V (Vista/Cámara), R (Reflejo)
    vec3 L = normalize(lightPosition.xyz - P);
    vec3 V = normalize(-P); // En View Space, la cámara está en (0,0,0),
    así que V es -P
}
```

```

    vec3 R = reflect(-L, N_norm);

    // 3. Componente Ambiental
    vec3 ambient = matAmbient.rgb;

    // 4. Componente Difusa (Lambert): N dot L
    vec3 diffuse = matDiffuse.rgb * max(0.0, dot(N_norm, L));

    // 5. Componente Especular (Phong): (R dot V)^shininess
    vec3 specular = matSpecular.rgb * pow(max(0.0, dot(R, V)),
matShininess);

    // 6. Suma final
    fragColor = vec4(ambient + diffuse + specular, 1.0);
}

```

Class C-Esqueleto (PER-GEOMETRY)

C-Vertex Shader

En este tipo de ejercicios, el Vertex Shader suele ser muy simple ("Pass-Through"). **No proyectamos** todavía a Clip Space porque el Geometry Shader necesita las coordenadas originales (Object Space) para hacer cálculos (como encontrar el centro de un triángulo).

```

#version 330 core

layout (location = 0) in vec3 vertex;
layout (location = 2) in vec3 color;

out vec4 vfrontColor;

void main()
{
    vfrontColor = vec4(color, 1.0);

    gl_Position = vec4(vertex, 1.0);
}

```

PROF

C-Geometry Shader

Este es el núcleo. Aquí es donde **transformas** las coordenadas a Clip Space y **emites** los vértices.

```

#version 330 core

layout(triangles) in;
layout(triangle_strip, max_vertices = 3) out;

```

```

// --- INPUTS (vienen del VS como arrays []) ---
in vec4 vfrontColor[];

// --- OUTPUTS (van al FS) ---
out vec4 gfrontColor;
// out vec2 gtexCoord;

// --- UNIFORMS ---
uniform mat4 modelViewProjectionMatrix;

void main( void )
{
    for(int i = 0; i < 3; i++)
    {
        gfrontColor = vfrontColor[i];
        vec4 pos = gl_in[i].gl_Position;
        gl_Position = modelViewProjectionMatrix * pos;
        EmitVertex(); // Emite 1 vértice
    }
    EndPrimitive(); // Cierra el triángulo
}

```

C-Fragment Shader

El Fragment Shader suele ser idéntico al de la **Class A**, solo que recibe los datos del GS (prefijo **g**) en lugar del VS.

```

#version 330 core

// --- INPUT (Viene del Geometry Shader) ---
in vec4 gfrontColor;
// in vec2 gtexCoord;

// --- OUTPUT ---
out vec4 fragColor;

void main()
{
    fragColor = gfrontColor;

    // Si tienes texturas o márgenes (bordes negros):
    // if (gtexCoord.s < 0.05 || gtexCoord.s > 0.95 ...) fragColor =
    vec4(0);
}

```

2. Receptes

A-Per Vertex

Aquí tens els tres snippets corregits i amb els conceptes clars.

He arreglat l'error greu del punt 2 (et faltava la matriu) i he reescrit el punt 3 perquè sigui realment un enviament en **Eye Space** (vital per a il·luminació), ja que el teu codi original barrejava conceptes.

Copia això a la teva secció **A-Per Vertex**.

A-Per Vertex

1. Passar al GS la posició, la normal, el color i la coordenada de textura de cada vèrtex (En Object Space)

Ús: Quan tens un **Geometry Shader** després que ha de fer càlculs físics (Lego, extrusions, explosions). El GS rebrà les dades "crues".

```
#version 330 core

layout (location = 0) in vec3 vertex;
layout (location = 1) in vec3 normal;
layout (location = 2) in vec3 color;
layout (location = 3) in vec2 texCoord;

out vec4 vfrontColor;
out vec3 vNormal;
out vec2 vtexCoord;

void main()
{
    // Passem els atributs tal qual (sense modificar)
    vfrontColor = vec4(color, 1.0);
    vNormal = normal;
    vtexCoord = texCoord;

    // Posició "crua" en Object Space.
    // El GS s'encarregarà de multiplicar per la matriu MVP després.
    gl_Position = vec4(vertex, 1.0);
}
```

2. Passar Posició en CLIP SPACE (Obligatori si NO hi ha GS)

Ús: Quan **NO tens Geometry Shader**, o quan el GS necessita coordenades de pantalla directament.

Important: Necessites la `modelViewProjectionMatrix`.

```
#version 330 core

layout (location = 0) in vec3 vertex;
uniform mat4 modelViewProjectionMatrix; // Imprescindible

void main()
{
    // Multipliquem per la matriu per projectar a la pantalla
    gl_Position = modelViewProjectionMatrix * vec4(vertex, 1.0);
}
```

3. Passar Dades en EYE SPACE (Preparat per Il·luminació)

Ús: Quan el següent pas (sigui GS o FS) ha de calcular **Il·luminació (Phong/Lambert)**. La llum es calcula sempre en Eye Space.

```
#version 330 core
layout (location = 0) in vec3 vertex;
layout (location = 1) in vec3 normal;

out vec3 vPosEye;    // Posició del vèrtex respecte la càmera
out vec3 vNormalEye; // Normal rotada correctament

uniform mat4 modelViewMatrix; // Per moure vèrtexs
uniform mat3 normalMatrix;    // Per rotar normals
uniform mat4 modelViewProjectionMatrix; // Per la posició final

void main() {
    // 1. Posició en Eye Space (per vectors L i V)
    vPosEye = (modelViewMatrix * vec4(vertex, 1.0)).xyz;

    // 2. Normal en Eye Space (per productes escalars)
    vNormalEye = normalize(normalMatrix * normal);

    // 3. La gl_Position depèn:
    // - Si hi ha GS: sol ser vec4(vertex, 1.0)
    // - Si NO hi ha GS: OBLIGATORI MVP * vertex (com aquí sota)
    gl_Position = modelViewProjectionMatrix * vec4(vertex, 1.0);
}
```

4. Passar la posició en Eye Space

// Si no hi ha GS s'envia en Eye space

```
out vec3 vNormal;      // Surt del VS
out vec2 vTexCoord;

void main() {
    // OBLIGATORI: Si no hi ha GS, el VS ha de calcular la posició final
    en CLIP SPACE
    gl_Position = modelViewProjectionMatrix * vec4(vertex, 1.0);

    vNormal = normal; // Envia dades per interpolar
}
```

B-Per Fragment

1. Assignar com a color del fragment el gris resultant d'utilitzar la component Z de la normal en eye space

```
#version 330 core

in vec3 gNormal;      // Normal en Eye Space (ve del GS)
// in vec4 gfrontColor; // En aquest cas, ignorem el color original
out vec4 fragColor;

void main() {
    // 1. NORMALITZAR (Vital!)
    // La interpolació entre vèrtexs escurça els vectors, cal
    normalitzar sempre al FS.
    vec3 N = normalize(gNormal);

    // 2. AGAFAR LA COMPONENT Z
    // En Eye Space, la Z positiva apunta cap a la càmera.
    // Això equival a fer N dot L, on L = (0,0,1).
    float intensity = N.z;

    // 3. PINTAR EN GRIS
    // Assignem la intensitat a R, G i B.
    fragColor = vec4(intensity, intensity, intensity, 1.0);
}
```

PROF

C-Per Geometry

1. Emetre un triangle corresponent al triangle original (amb el color sense il·luminació)

```

for (int i = 0; i < 3; i++) {
    gfrontColor = vfrontColor[i];

    // Transform Object Space -> Clip Space
    vec4 pos = gl_in[i].gl_Position; // 1. Get Raw Object Position
    gl_Position = modelViewProjectionMatrix * pos; // D. Clip Space
(Projection
    EmitVertex();
}
EndPrimitive();

```

// Solució alternativa

```

for (int i = 0; i < 3; i++) {
    gfrontColor = vfrontColor[i];

    // Transform Object Space -> Clip Space
    vec3 pos = gl_in[i].gl_Position.xyz; // 1. Get Raw Object
Position
    gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0); //
D. Clip Space (Projection
    EmitVertex();
}
EndPrimitive();

```

2. Emetre un triangle corresponent a la projecció del triangle original al pla Y anterior (Shadow)

```

const vec4 COLOR_BLACK = vec4(0.0, 0.0, 0.0, 1.0);
for( int i = 0 ; i < 3 ; i++ )
{
    gfrontColor = COLOR_BLACK;
    vec3 shadowPos = gl_in[i].gl_Position.xyz;
    shadowPos.y = boundingBoxMin.y;

    gl_Position = modelViewProjectionMatrix * vec4(shadowPos, 1.0);
    EmitVertex();
}

```

3. Si és el 1r triangle (està processant la primera primitiva del objecte), dibuixar un rectangle centrat en Centre de costat Longitud, 0.01 per sota de boundingBoxMin.y

Per aquest exemple, donarem el cas on:

- Pintar el rectangle cyan
- Rectangle de costat $2R$, on R és la meitat de la diagonal de la caps englobant de l'escena
- Centrat en $(C.x, boundingBoxMin.y, C.z)$, on C és el centre de la caps englobant de l'escena

```

if (gl_PrimitiveIDIn == 0) {
    // Calcular Centre i Longitud
    // "Sigui R la meitat de la diagonal... i sigui C el centre"
    vec3 C = vec3(boundingBoxMax + boundingBoxMin) / 2.0;
    float R = length(boundingBoxMax - boundingBoxMin) / 2.0;

    // "0.01 per sota de boundingBoxMin.y"
    vec3 Center = vec3(C.x, boundingBoxMin.y - 0.01, C.z);
    float Longitud = 2*R;

    // Rectangle de color Cyan
    const vec4 CYAN = vec4(0.0, 1.0, 1.0, 1.0);
    glFrontColor = CYAN;

    // --- 2. DEFINICIÓ BASE (Centrat a 0,0,0) ---
    // Creem un quadrat UNITARI (costat 1.0) centrat a l'origen.
    // Així, quan multipliquem per 'Longitud', tindrà exactament la
mida 'Longitud'.
    // Ordre: Triangle Strip (Baix-Esq, Baix-Dreta, Dalt-Esq, Dalt-
Dreta)
    vec3 offsets[4];
    offsets[0] = vec3(-0.5, 0.0, -0.5);
    offsets[1] = vec3( 0.5, 0.0, -0.5);
    offsets[2] = vec3(-0.5, 0.0,  0.5);
    offsets[3] = vec3( 0.5, 0.0,  0.5);

    // Pels 4 vèrtexs del quadrat
    for (int i = 0; i < 4; i++) {
        vec3 pos = offsets[i]; // 1. CENTRAR (Ja està al 0,0,0
local)

        // 2. ESCALAR (Multipliquem per la mida final desitjada)
        // De mida 1 passa a mida 'Longitud' (que és 2*R)
        pos = pos * Longitud;

        // 3. ROTAR (No cal aquí)

        // 4. DESCENTRAR (No cal perquè el nostre origen local era
correcte)

        // 5. MOURE (Translació final a la posició de destí)
        pos = pos + Center;

        // Sortida
        gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0);
        EmitVertex();
    }
}

```



```
    EndPrimitive();  
}
```

4. Aplicar a cada vèrtex del triangle la translació T

Nota: Per fer la translació no cal estar centrat: ens ho saltem

```
// Suposem que tenim un vector T definit, per exemple:  
// vec3 T = vec3(0.0, 1.0, 0.0);  
  
for(int i = 0; i < 3; i++)  
{  
    gfrontColor = vfrontColor[i];  
  
    // Obtenir posició original (Object Space)  
    vec3 pos = gl_in[i].gl_Position.xyz;  
  
    // 5. MOURE (Translació T)  
    // Simplement sumem el vector al vèrtex actual  
    pos = pos + T;  
  
    // Projecció final  
    gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0);  
    EmitVertex();  
}  
EndPrimitive();
```

5. Calcular n, on n és el promig de les normals dels 3 vèrtexs del triangle en Object Space

```
in vec3 vNormal[];  
  
void main ( void() ) {  
    vec3 n = (vNormal[0] + vNormal[1] + vNormal[2]) / 3.0;  
    ...  
}
```

//EXTRA, si ens demanen el promig normalitzat

```
in vec3 vNormal[];  
  
void main ( void() ) {  
    //Nota dividir per 3 si fas normalize es redundant matemàticament  
    vec3 n = normalize(vNormal[0] + vNormal[1] + vNormal[2]);  
}
```

```
} ...
```

6. Treure els vèrtexs en Clip Space (gl_Position)

Nota: Sempre ens demanaran que treguem els vèrtexs en Clip Space el GS

```
for(int i = 0; i < 3; i++)
{
    gfrontColor = vfrontColor[i];
    vec3 pos = gl_in[i].gl_Position.xyz;
    gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0);
    EmitVertex();
}
EndPrimitive();
```

7. Aplicar una translació en Eye Space

Quan t'ho demanen?

Quan la translació depèn de la càmera.

- Exemple: "Mou els vèrtexs 2 unitats **cap a la càmera**".
- Exemple: "Fes que l'objecte sempre estigui orientat cap a l'observador."

```
// Vector T en Eye Space (Ex: moure 1 unitat cap a la dreta de la
PANTALLA)
vec3 T_eye = vec3(1.0, 0.0, 0.0);

for(int i = 0; i < 3; i++) {
    // 1. OBJECT -> EYE
    vec4 posEye = modelViewMatrix * gl_in[i].gl_Position;

    // 2. APLICAR TRANSLACIÓ (En coordenades de càmera)
    posEye.xyz = posEye.xyz + T_eye;

    // 3. EYE -> CLIP (Només matriu Projectió, la ModelView ja l'hem
usat)
    gl_Position = projectionMatrix * posEye;

    EmitVertex();
}
```

PROF

8. Aplicar a cada vèrtex del triangle, una translació T, una rotació R i un escalat S

```

for(int i = 0; i < 3; i++)
{
    gfrontColor = vfrontColor[i];
    vec3 pos = gl_in[i].gl_Position.xyz;

    // --- ELS 5 PASSOS UNIVERSALS ---

    // 1. CENTRAR (Portar l'objecte al 0,0,0 local)
    pos = pos - C;

    // 2. ESCALAR (Canviar la mida)
    pos = pos * S;

    // 3. ROTAR (Aplicar matriu de rotació)
    pos = R * pos;

    // 4. DESCENTRAR (Tornar a la posició original)
    pos = pos + C;

    // 5. MOURE (Aplicar la translació final T)
    pos = pos + T;

    // -----

    gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0);
    EmitVertex();
}
EndPrimitive();

```

9. Calcular el centre d'un triangle (Baricentre)

```

vec3 C = (gl_in[0].gl_Position.xyz + gl_in[1].gl_Position.xyz +
gl_in[2].gl_Position.xyz) / 3.0;

```

10. Dibuixar n triangles (primitives) per segon

Anem a assumir que volem que el GS emeti només els n primers triangles. Però si n vé donat pel temps t, anirem dibuixant més triangles cada segon fins haver dibuixat la figura sencera.

Suposem que volem dibuixar els n primers triangles on $n = 100 \cdot \text{time}$ (truncament).

```

uniform float time;
int n = floor(100.0*time);

if (gl_PrimitiveIDIn <= n) {

```

```

    for (int i = 0; i < 3; i++) {
        gfrontColor = vfrontColor[i];

        // Transform Object Space -> Clip Space
        vec3 pos = gl_in[i].gl_Position.xyz; // 1. Get Raw Object
Position
        gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0); //
D. Clip Space (Projection
        EmitVertex();
    }
    EndPrimitive();
}

```

11. Emetre els triangles amb il.luminació (enviar normal i posició al FS amb Eye Space)

Nota: GS no calcula la il.luminació, ho prepara perquè fragment shader la calculi.

```

// --- CONFIGURACIÓ ---
in vec3 vNormal[]; // Normal que ve del Vertex Shader (Object Space)
out vec3 gNormal; // Normal cap al Fragment Shader (Eye Space)
out vec3 gPos; // Posició cap al Fragment Shader (Eye Space)

uniform mat4 modelViewMatrix;
uniform mat3 normalMatrix;

void main() {
    for (int i = 0; i < 3; i++) {
        // 1. NORMAL: Transformem a Eye Space per càlculs de llum
        // (Important: utilitzar normalMatrix)
        gNormal = normalize(normalMatrix * vNormal[i]);

```

```

        // 2. POSICIÓ (Física): Transformem a Eye Space
        // (Necessari per calcular el vector Llum L i Visió V al FS)
        gPos = (modelViewMatrix * gl_in[i].gl_Position).xyz;

        // 3. POSICIÓ (Pantalla): Transformem a Clip Space
        // (Això és només per pintar el píxel al lloc correcte)
        gl_Position = modelViewProjectionMatrix * gl_in[i].gl_Position;

        EmitVertex();
    }
    EndPrimitive();

```

```

}

```

12. Emetre un cub de costat Longitud centrat al punt més proper al baricentre del triangle

Nota: Si dividim l'espai en cubs de mida Longitud, el centre de qualsevol cub es troba fent: $\text{round}(\text{Posicio} / \text{Longitud}) * \text{Longitud}$.

```
// --- 1. Càlcul del centre i mida ---
    vec3 BT = (gl_in[0].gl_Position.xyz + gl_in[1].gl_Position.xyz +
gl_in[2].gl_Position.xyz) / 3.0;

    // Suposem que la Longitud ens la donen o la calculem (ex: constant)
    float Longitud = 0.5; // 0 un uniform, o 2*R...

    // "Punt més proper al baricentre" (Quantització / Snap to Grid)
    // Això troba el centre del "voxel" on cau el triangle
    vec3 Center = round(BT / Longitud) * Longitud;

// --- 2. Geometria base (Cub Unitari al 0,0,0) ---
// Definim els 8 vèrtexs del cub unitari (-0.5 a 0.5)
vec3 v[8];
v[0] = vec3(-0.5, -0.5, -0.5);
v[1] = vec3( 0.5, -0.5, -0.5);
v[2] = vec3(-0.5,  0.5, -0.5);
v[3] = vec3( 0.5,  0.5, -0.5);
v[4] = vec3(-0.5, -0.5,  0.5);
v[5] = vec3( 0.5, -0.5,  0.5);
v[6] = vec3(-0.5,  0.5,  0.5);
v[7] = vec3( 0.5,  0.5,  0.5);

// Definim les 6 cares utilitzant índexs (Triangle Strips)
// Cada fila és una cara (4 vèrtexs en ZIG-ZAG)
int faces[6][4] = int[][](
    int[](4, 5, 6, 7), // Front (+Z)
    int[](1, 0, 3, 2), // Back (-Z)
    int[](2, 3, 6, 7), // Top (+Y)
    int[](4, 5, 0, 1), // Bottom (-Y)
    int[](5, 1, 7, 3), // Right (+X)
    int[](0, 4, 2, 6)  // Left (-X)
);

// --- 3. Bucle de dibuixat (6 cares) ---
const vec4 GREY = vec4(0.8);

for (int f = 0; f < 6; f++) {
    gfrontColor = GREY; // 0 un color diferent per cara

    // Per a cada vèrtex de la cara (Strip de 4)
    for (int i = 0; i < 4; i++) {
        // Obtenim el vèrtex base unitari
        int idx = faces[f][i];
        vec3 pos = v[idx];
```

```

// --- ELS 5 PASSOS UNIVERSALS ---

// 1. CENTRAR: (Ja està centrat a l'origen local 0,0,0)

// 2. ESCALAR: Multipliquem per la mida desitjada
// (El cub unitari fa 1.0, ara farà 'Longitud')
pos = pos * Longitud;

// 3. ROTAR: (Opcional, aquí no en demanen)

// 4. DESCENTRAR: (No cal)

// 5. MOURE: Portar al centre calculat
pos = pos + Center;

// --- PROJECCIÓ ---
gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0);
EmitVertex();
}
EndPrimitive(); // Tanquem la tira després de cada cara
(important!)
}

```

13. Emetre un cub de costat Longitud centrat al punt més proper al baricentre del triangle, pintant el cub del color més proper en distància Euclídea al color del vèrtex (a una llista de colors ja definida).

```

#version 330 core

layout (triangles) in;
layout (triangle_strip, max_vertices = 24) out;

in vec4 vfrontColor[];
out vec4 gfrontColor;

uniform mat4 modelViewProjectionMatrix;

// --- DEFINICIÓ DE LA PALETA I FUNCIO AUXILIAR ---
const vec4 PALETTE[5] = vec4[](
    vec4(1.0, 0.0, 0.0, 1.0), // Vermell
    vec4(0.0, 1.0, 0.0, 1.0), // Verd
    vec4(0.0, 0.0, 1.0, 1.0), // Blau
    vec4(0.0, 1.0, 1.0, 1.0), // Cyan
    vec4(1.0, 1.0, 0.0, 1.0)  // Groc
);

vec4 findNearestColor(vec4 inputColor) {
    vec4 bestColor = PALETTE[0];

```

```

float minDist = 1000.0;

for (int i = 0; i < 5; i++) {
    float d = distance(inputColor, PALETTE[i]);
    if (d < minDist) {
        minDist = d;
        bestColor = PALETTE[i];
    }
}
return bestColor;
}

void main() {
    // --- 1. CÀLCUL DEL CENTRE I MIDA ---
    vec3 BT = (gl_in[0].gl_Position.xyz + gl_in[1].gl_Position.xyz +
gl_in[2].gl_Position.xyz) / 3.0;

    // Suposem que la Longitud ens la donen o la calculem (ex: constant)
    float Longitud = 0.5; // (Aquest és el teu 'step')

    // "Punt més proper al baricentre" (Quantització / Snap to Grid)
    vec3 Center = round(BT / Longitud) * Longitud;

    // --- CÀLCUL DEL COLOR (NOU) ---
    // Calculem el promig i busquem el més proper a la llista
    vec4 avgColor = (vfrontColor[0] + vfrontColor[1] + vfrontColor[2]) /
3.0;
    vec4 FinalColor = findNearestColor(avgColor);

    // --- 2. GEOMETRIA BASE (Cub Unitari al 0,0,0) ---
    vec3 v[8];
    v[0] = vec3(-0.5, -0.5, -0.5);
    v[1] = vec3( 0.5, -0.5, -0.5);
    v[2] = vec3(-0.5,  0.5, -0.5);
    v[3] = vec3( 0.5,  0.5, -0.5);
    v[4] = vec3(-0.5, -0.5,  0.5);
    v[5] = vec3( 0.5, -0.5,  0.5);
    v[6] = vec3(-0.5,  0.5,  0.5);
    v[7] = vec3( 0.5,  0.5,  0.5);

    // Definim les 6 cares utilitzant índexs (Triangle Strips)
    int faces[6][4] = int[][](
        int[](4, 5, 6, 7), // Front (+Z)
        int[](1, 0, 3, 2), // Back (-Z)
        int[](2, 3, 6, 7), // Top (+Y)
        int[](4, 5, 0, 1), // Bottom (-Y)
        int[](5, 1, 7, 3), // Right (+X)
        int[](0, 4, 2, 6)  // Left (-X)
    );

    // --- 3. BUCLE DE DIBUIXAT (6 CARES) ---

    for (int f = 0; f < 6; f++) {

```

```

    gfrontColor = FinalColor; // Assignem el color calculat

    // Per a cada vèrtex de la cara (Strip de 4)
    for (int i = 0; i < 4; i++) {
        // Obtenim el vèrtex base unitari
        int idx = faces[f][i];
        vec3 pos = v[idx];

        // --- ELS 5 PASSOS UNIVERSALS ---

        // 1. CENTRAR: (Ja està centrat a l'origen local 0,0,0)

        // 2. ESCALAR: Multipliquem per la mida desitjada
        // (El cub unitari fa 1.0, ara farà 'Longitud')
        pos = pos * Longitud;

        // 3. ROTAR: (Opcional, aquí no en demanen)

        // 4. DESCENTRAR: (No cal)

        // 5. MOURE: Portar al centre calculat
        pos = pos + Center;

        // --- PROJECCIÓ ---
        gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0);
        EmitVertex();
    }
    EndPrimitive(); // Tanquem la tira després de cada cara
    (important!)
}
}
}

```

14. Emetre un cub de costat Longitud centrat al punt més proper al baricentre del triangle, assignant-li una textura a la cara superior

PROF

```

#version 330 core

layout (triangles) in;
layout (triangle_strip, max_vertices = 24) out;

in vec4 vfrontColor[];

// OUTS cap al Fragment Shader
out vec4 gfrontColor;
out vec2 gTexCoord; // Coordenades de textura (s, t)
flat out int isTop; // FLAG: 1 si és Top, 0 si no. (FLAT = sense
interpolar)

uniform mat4 modelViewProjectionMatrix;

```



```

void main() {
    // --- 1. Càlcul del centre i mida (Igual que abans) ---
    vec3 BT = (gl_in[0].gl_Position.xyz + gl_in[1].gl_Position.xyz +
gl_in[2].gl_Position.xyz) / 3.0;

    float Longitud = 0.5; // Constant o Uniform
    vec3 Center = round(BT / Longitud) * Longitud; // Lego Snap

    // --- 2. Definició del cub ---
    vec3 v[8];
    v[0] = vec3(-0.5, -0.5, -0.5); v[1] = vec3( 0.5, -0.5, -0.5);
    v[2] = vec3(-0.5,  0.5, -0.5); v[3] = vec3( 0.5,  0.5, -0.5);
    v[4] = vec3(-0.5, -0.5,  0.5); v[5] = vec3( 0.5, -0.5,  0.5);
    v[6] = vec3(-0.5,  0.5,  0.5); v[7] = vec3( 0.5,  0.5,  0.5);

    int faces[6][4] = int[][](
        int[](4, 5, 6, 7), // 0: Front
        int[](1, 0, 3, 2), // 1: Back
        int[](2, 3, 6, 7), // 2: TOP (+Y) -> La important
        int[](4, 5, 0, 1), // 3: Bottom
        int[](5, 1, 7, 3), // 4: Right
        int[](0, 4, 2, 6)  // 5: Left
    );

    // Coordenades de textura per a un quadrat (ordre Triangle Strip:
    ZIG-ZAG)
    // 0: Baix-Esq, 1: Baix-Dreta, 2: Dalt-Esq, 3: Dalt-Dreta
    vec2 quadUV[4] = vec2[](
        vec2(0.0, 0.0),
        vec2(1.0, 0.0),
        vec2(0.0, 1.0),
        vec2(1.0, 1.0)
    );

    const vec4 GREY = vec4(0.8, 0.8, 0.8, 1.0);

    // --- 3. Bucle d'emissió ---
    for (int f = 0; f < 6; f++) {

        // Determinem si som a la cara TOP (índex 2)
        int currentIsTop = 0;
        if (f == 2) currentIsTop = 1;

        for (int i = 0; i < 4; i++) {
            // OUT 1: Flag (flat)
            isTop = currentIsTop;

            // OUT 2: Color (Sempre Gris)
            gfrontColor = GREY;

            // OUT 3: Textura
            // Només té sentit si isTop==1, però passem el valor sempre

```

```

per evitar errors
    gTexCoord = quadUV[i];

    // Càlcul posició
    vec3 pos = v[faces[f][i]];
    pos = pos * Longitud; // Escalar
    pos = pos + Center;   // Moure

    gl_Position = modelViewProjectionMatrix * vec4(pos, 1.0);
    EmitVertex();
}
EndPrimitive();
}
}

```

// El FS quedaria així:

```

#version 330 core

// INPUTS (Han de coincidir amb el GS)
in vec4 gfrontColor;
in vec2 gTexCoord;
flat in int isTop; // Rebem l'enter sense interpolar

out vec4 fragColor;

uniform sampler2D colorMap; // La textura (ex: una cara de Lego, una
                             caixa, etc.)

void main() {
    if (isTop == 1) {
        // --- OPCIÓ A: Textura pura (La foto tal qual) ---
        // El color final és exactament el de la imatge. El gfrontColor
        s'ignora.
        fragColor = texture(colorMap, gTexCoord);

        // --- OPCIÓ B: Barreja/Tintat (Multiplicació) ---
        // Si la textura és blanca, es veu el gfrontColor.
        // Si la textura té color, es barreja (ex: Blau * Vermell =
        Negre/Lila).
        // fragColor = texture(colorMap, gTexCoord) * gfrontColor;

        // --- OPCIÓ C: Màscara d'Intensitat (Ignorar color de la
        textura) ---
        // Fem servir la imatge només per donar "llum" o "forma", però
        forcem
        // que el to sigui el del gfrontColor. Ideal si la textura és
        una peça de Lego
        // gris/blanca i la vols pintar de colors.
        // float intensity = texture(colorMap, gTexCoord).r; // Usem
        només el canal Vermell com a brillantor
    }
}

```

```

        // fragColor = vec4(gfrontColor.rgb * intensity, gfrontColor.a);
    }
    else {
        // Si és qualsevol altra cara, pintem el color gris base
        fragColor = gfrontColor;
    }
}

```

15. Emetre un prisma de base triangular amb altura $N \cdot d$ (sigui d un uniform definit per l'usuari)

```

#version 330 core

layout (triangles) in;
// 3 (Base) + 3 (Tapa) + 12 (3 parets x 4 vèrtexs) = 18 vèrtexs mínim
layout (triangle_strip, max_vertices = 20) out;

in vec4 vfrontColor[];
out vec4 gfrontColor;

uniform mat4 modelViewProjectionMatrix;
uniform float d; // Altura de l'extrusió (definit per l'usuari)

void main() {
    // --- 1. CÀLCUL DE LA DIRECCIÓ D'EXTRUSIÓ (Normal) ---
    // Vectors de les arestes del triangle original
    vec3 V0 = gl_in[0].gl_Position.xyz;
    vec3 V1 = gl_in[1].gl_Position.xyz;
    vec3 V2 = gl_in[2].gl_Position.xyz;

    vec3 edge1 = V1 - V0;
    vec3 edge2 = V2 - V0;

    // Normal del triangle (direcció perpendicular)
    vec3 N = normalize(cross(edge1, edge2));

    // Vector de desplaçament (Altura)
    vec3 Offset = N * d;

    // --- 2. DIBUIXAR LA BASE (Triangle Original) ---
    // Normalment la base mira "cap avall", però aquí la farem tal qual
    gfrontColor = vfrontColor[0]; // Color base

    gl_Position = modelViewProjectionMatrix * vec4(V0, 1.0);
    EmitVertex();
    gl_Position = modelViewProjectionMatrix * vec4(V1, 1.0);
    EmitVertex();
    gl_Position = modelViewProjectionMatrix * vec4(V2, 1.0);
    EmitVertex();
}

```

```

EndPrimitive();

// --- 3. DIBUIXAR LA TAPA (Triangle Desplaçat) ---
// Pintem la tapa una mica més clara per efecte visual (opcional)
gfrontColor = vfrontColor[0];

    gl_Position = modelViewProjectionMatrix * vec4(V0 + Offset, 1.0);
EmitVertex();
    gl_Position = modelViewProjectionMatrix * vec4(V1 + Offset, 1.0);
EmitVertex();
    gl_Position = modelViewProjectionMatrix * vec4(V2 + Offset, 1.0);
EmitVertex();
    EndPrimitive();

// --- 4. DIBUIXAR LES PARETS LATERALS (3 Quads) ---
// Connectem cada aresta de la base amb l'aresta de la tapa
gfrontColor = vfrontColor[0];

for (int i = 0; i < 3; i++) {
    int next = (i + 1) % 3; // Índex del següent vèrtex (0->1, 1->2,
2->0)

    vec3 v_base_curr = gl_in[i].gl_Position.xyz;
    vec3 v_base_next = gl_in[next].gl_Position.xyz;

    vec3 v_top_curr = v_base_curr + Offset;
    vec3 v_top_next = v_base_next + Offset;

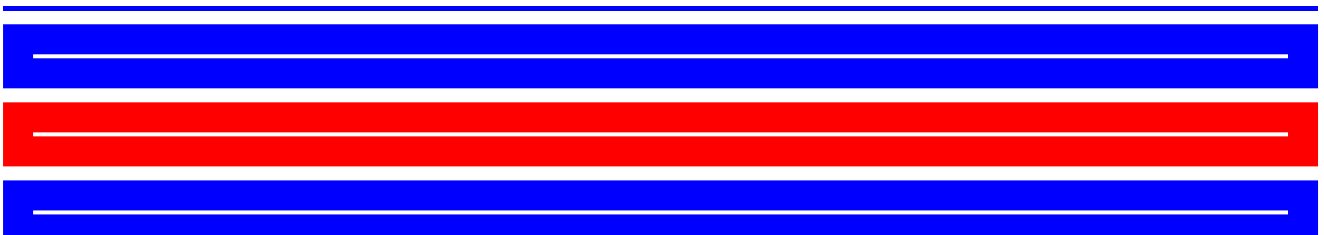
    // Dibuixem un QUAD (Rectangle) usant Triangle Strip (Zig-Zag)
    // Ordre: Base1 -> Base2 -> Top1 -> Top2
    gl_Position = modelViewProjectionMatrix * vec4(v_base_curr,
1.0); EmitVertex();
    gl_Position = modelViewProjectionMatrix * vec4(v_base_next,
1.0); EmitVertex();
    gl_Position = modelViewProjectionMatrix * vec4(v_top_curr,
1.0); EmitVertex();
    gl_Position = modelViewProjectionMatrix * vec4(v_top_next,
1.0); EmitVertex();

    EndPrimitive(); // Tanquem cada paret individualment
}
}

```

PROF

16.



3. Exercises

A-Per Vertex

B-Per Fragment

C-Per Geometry